Fate's Edge: A Deep Dive into Mechanics and Design Philosophy

For the analytically minded player who wants to understand why the dice fall the way they do

---

1 The Core Philosophy: Beyond Success/Failure

Most RPGs reduce outcomes to a binary: did you succeed or fail? Fate's Edge rejects this. Instead, it asks: What happens next? This single question reshapes every mechanic.

The design mantra: Every roll must move the story forward—no matter the result. Even a "failure" creates new opportunities, while success often introduces complications. This eliminates dead ends and ensures the narrative constantly evolves.

Why this matters to strategists: The system transforms dice from pass/fail meters into narrative catalysts. Your goal isn't to maximize success rate—it's to manage the flow of consequences so that both triumphs and setbacks drive meaningful story.

2 The Dice Engine: d10s with Purpose

2.1 How Rolls Work

- Roll a pool of d10s: Attribute (1-5) + Skill (0-5)
- Each die showing 6-10 = 1 success; a natural 10 = 2 successes
- Target number: Difficulty Value (DV) from 2 (routine) to 5+ (extreme). Default DV = 3

2.2 Why d10s?

- Clean 50% success chance per die (6-10 = 5/10 faces)
- Enables quick mental math: 4 dice 50% chance of at least one 10
- The "10 = 2" mechanic creates meaningful variance without extra rolls

2.3 Key Probability Insight

With 4 dice (a common sweet spot):

- Chance of at least one success: 93.75% (only 6.25% miss rate)
- Chance of at least one 10: 34.4% (triggers bonus effects)

- Chance of rolling a 1 (for GM): 34.4% (gives the GM narrative leverage)

Strategic implication: The first points in attributes/skills give huge returns (going from 2→3 dice cuts miss rate from 25%→12.5%). Beyond 4 dice, diminishing returns kick in sharply—miss rate drops from 6.25%→3.9% for 5 dice, but GM leverage increases significantly. Smart players cap raw dice at 4 and use positioning to boost effective pool size.

3 Position: The Tactical Lever

Position (Dominant/Controlled/Desperate) doesn't just modify difficulty—it reshapes your probability distribution before you roll:

Table 1: Position Effects

Position	Mechanical Effect
Dominant	Re-roll one failure (die <6)
Controlled	No re-rolls
Desperate	Re-roll one success (die 6+)

What this does mathematically:

- Dominant: Cuts the left tail of your success distribution. Reduces chance of zero successes dramatically (e.g., 25%→6.25% at 2 dice).
- Desperate: Increases volatility—more likely to get extreme results (0 or 2+ successes). Favors high-skill players who can turn a re-rolled 6 into a 10.
- Critical nuance: Position adjustments stack as dice. If already Dominant, further improvements become +1 die (not additional re-rolls). This prevents abuse while keeping positioning meaningful.

Why not adjust DV? Because position reflects narrative risk, not raw task difficulty. Picking a lock while guards approach (Desperate) feels different than in a safe room (Dominant)—even if the DV is identical. This forces players to engage with fiction tactically.

4 The Outcome Matrix: Four Paths Forward

Every roll lands in one of four states—all create narrative momentum:

Why this eliminates dead ends:

- A Partial isn't failure—it's meaningful progress (e.g., "The lock is stiff but you think you can get it if you keep trying").
- A Miss generates resources (Boons) that let you influence the next roll. The

Table 2: Outcome Matrix

Result	Outcome
Successes $\geq$ DV, SB $= 0$	Clean Success
	Intent achieved, no added friction
Successes $\geq$ DV, SB > 0	Success with SB
	Intent achieved; GM spends SB for complications
$0 <$ Successes $<$ DV	Partial
	Progress made; gain 1 Boon
Successes $= 0$	Miss
	No progress; gain 2 Boons; GM spends SB

GM must spend SB to complicate the situation—nothing is ever "nothing happens."

Strategic depth: The system creates a resource loop:

- Boons (from Partial/Miss) let you re-roll dice or improve position
- Story Beats (SB) (from 1s rolled) let the GM introduce complications
- Smart players embrace small failures early to stockpile Boons for critical moments—but Boons cap at 5 and reset partially between scenes, preventing hoarding.

5 Resources: Boons (Player) and Story Beats (GM)

5.1 Boons: Your Tactical Reserve

Boons represent momentum, luck, and narrative leverage. Spend them to:

- Re-roll one die (1 Boon)
- Improve position by one step before rolling (1 Boon)
- Activate an off-screen asset (1 Boon)
- Convert 2 Boons $\rightarrow$ 1 XP (once per session, max 2 XP/session)

The XP conversion trap: Converting Boons to XP is rarely optimal. One Boon via re-roll gives 0.4 expected successes—often worth more than the XP gained from those successes. Only convert when:

- You're at the 5-Boon cap (about to lose excess)
- The session is ending (Boons reset to 2)

Optimal use: Always keep 1 Boon for critical re-rolls. Spending 1 Boon to improve position (e.g., Controlled$\rightarrow$Dominant) before a big roll is often better than re-rolling

after—it affects your entire pool.

5.2 Story Beats: GM's Narrative Fuel

Every 1 rolled gives the GM 1 Story Beat. Spend SB to:

- 1 SB: Minor complication (noise, distraction, +1 clock segment)
- 2 SB: Moderate complication (alarm raised, lose advantage, worsen position)
- 3+ SB: Major complication (reinforcements, scene shift, broken asset)

The high-level shift: At lower tiers, the GM spends SB sparingly (1-2/scene). At tiers III+ (90+ XP), players routinely roll 6-8 dice—generating 0.47-0.57 expected SB per roll. In a 4-player round, that's 2-2.3 SB generated. Smart GMs spend this aggressively:

> "You pick the lock cleanly—but the noise alerted the patrol. Reinforcements [6] tick +2."
> "You disarm the trap—but the recoil jars your aim. Your next shot is at -1 die."

Result: Even successes create meaningful complications. Players learn: Success means the world reacted—not that you're safe.

6 Harm and Fatigue: The Anti-Death Spiral

Most games punish injury with worsening rolls—a death spiral. Fate's Edge does the opposite:

- Fatigue: Tracks exertion (max = Body). Each point worsens position (Dominant→→Desperate). If already Desperate, each fatigue = -1 die.
- Harm: Injury severity (1=-1 die on related actions, 2=-2 dice on most actions, 3=incapacitated).

The key innovation: Taking harm clears all fatigue. The shock of a wound resets exhaustion—creating dramatic swings (e.g., a fatigued character who takes harm suddenly finds second wind).

Armor's role: Armor converts harm to fatigue:

Table 3: Armor Conversion

Armor	Harm 1	Harm 2	Harm 3
Light	Fatigue 1	Harm 1 + Fatigue 1	Harm 2 + Fatigue 1
Medium	Fatigue 1	Fatigue 1	Harm 2 + Fatigue 1
Heavy	Fatigue 1	Fatigue 1	Fatigue 2

Strategic implication: Heavy armor turns lethal blows into manageable exhaustion—but is less efficient against multiple small hits. Smart players:

- Use heavy armor against expected high-damage threats
- Accept light/medium armor for versatile threat environments
- Sometimes seek low-risk harm (e.g., trapping a hand in a door) to reset fatigue when overwhelmed

7 Clocks: Visible Tension Tracking

Clocks track progress toward an event (4-10 segments). They advance via:

- Misses/Partials (often +1 segment)
- GM spending SB (usually 1 segment per SB)
- Player actions (e.g., successful skill checks)

Why they matter strategically: Clocks turn abstract threats into tangible resources. A "Barrow Collapse [6]" clock means:

- You have 6 "safe" actions before the entrance seals
- Each failed roll or GM SB spend brings you closer to disaster
- Smart players treat clocks like countdown timers—allocating actions to delay or accelerate based on goals

High-level insight: At higher tiers, clocks become the primary threat meter. The GM doesn't just add complications—they use clocks to make pressure visible and inevitable. Players must manage clock state alongside their resources.

8 Magic: The TAGS System

Free casting uses TAGS (keywords) to build spells. DV is calculated simply:

$$DV = 1 + numberofTAGS$$

with dangerous TAGS (Leap, Fold, Gate, Transform, Dominate) adding +2 DV each.

8.1 Example Spells

Critical clarification:

- [Air][Defend]: Pure elemental build (DV 3)
- [Ward]: Uses ward mechanics (DV = [Ward]'s base, usually 3) creates an integrity clock—the shield can take hits before failing. This is superior for sustained defense—[Air][Defend] is one-time only.

Table 4: TAGS Examples

Spell	TAGS	DV
Light	[Illuminate]	2
Heal	[Heal]	2
Shield of Air	[Air][Defend]	3
Fire Wall	[Fire][Wall]	3
Teleport	[Fold] (dangerous)	1+2=3

Why this system wins:

- Transforms spell design into a cost-benefit equation
- Rewards creativity within clear constraints (e.g., "Firebolt" = [Fire] DV 2; add [Ranged] for distance = DV 3)
- Scales risk: More TAGS = higher DV = more chance of backlash
- Backlash is thematic (fire backlash = smoke→blaze; earth = dust→fissure)

Smart caster play: Build minimum TAGS for effect, then:

- Use positioning to improve effective DV
- Accept manageable backlash (e.g., fatigue 1) for high-impact spells
- Save high-risk spells for moments when you can mitigate consequences

9 Advancement: The XP Economy

XP costs scale to encourage meaningful choices:

Table 5: XP Costs

Improvement	Cost	Downtime
Attribute increase	New rating × 3	New rating in days
Skill increase	New level × 2	New level in days
Minor Asset (tool, contact)	4 XP	1 day
Standard Asset (safehouse, network)	8 XP	1 week
Major Asset (fortress, guildhall)	12 XP	1 month
Minor Talent (edge case)	2-4 XP	3-10 days
Major Talent (core ability)	5-8 XP	1-2 weeks

Design rationale:

- Attributes cost more because they affect multiple skills (e.g., high Body aids melee, athletics, stealth)

- Skills cost less because they're narrower in application
- Creates natural progression: Broad attribute foundation → Skill specialization → Talent mastery

Diminishing returns in action:

- Raising an attribute from 4→5: 15 XP for +1 die on 3-5 skills
- Raising a skill from 4→5: 10 XP for +1 die on only that skill
- Optimal path: Max key attributes to 4 first (60 XP for all four at 4), then specialize skills to 3-4, then take talents

Tiers of play (XP ranges):

- Novice (0-40): Local reputation, prestige locked
- Seasoned (41-90): Regional notice, prestige abilities unlock
- Veteran (91-150): National influence, second follower slot suggested
- Paragon (151-220): Movers and shakers, rivals emerge
- Mythic (221+): Legendary status, kingdoms respond

At Veteran+: You'll regularly roll 8+ dice. The game responds not by inflating DV, but by making the GM's SB spend more impactful—reinforcements arrive faster, complications cascade, and success requires meaningful sacrifice.

10 Design Philosophy Summary

Table 6: Core Design Principles

Goal	Mechanic
No wasted rolls	4-state outcome matrix (always creates narrative)
Failure drives story	Miss generates Boons; GM spends SB
Risk vs. reward	Larger dice pools = more success and more 1s (GM leverage)
Tactical positioning	Position reshapes probability distribution pre-roll
No death spiral	Harm clears fatigue
Visible tension	Clocks track progress toward events
Creative magic	TAGS system: DV = 1 + TAGS (dangerous +2)
Balanced advancement	Attribute cost > skill cost; clear XP tiers

What this game is not:

- It's not simulationist grit (e.g., tracking calories)
- It's not a zero-sum tactical wargame (e.g., chess with dice)
- It is a narrative-first system with transparent, optimizable mechanics—where

the dice are inputs to a resource-driven story engine.

Final strategist's tip: The most powerful upgrade isn't more dice—it's effective dice via positioning. Spending 1 Boon to improve position (Controlled→Dominant) before a roll is equivalent to +2 dice in miss reduction—but costs 0 XP. Save your XP for attributes that cover multiple skills, then use positioning and Boons to hit the 4-5 effective dice sweet spot where miss rates plummet but GM leverage stays manageable.

A Verification Script

This Python script validates the probability claims in the analysis. Run it to see the math behind the 4-dice sweet spot and GM SB scaling.

```
import numpy as np

def simulate_dice_pool(num_dice, trials=100000):
    """Simulate rolling num_dice d10s and return statistics."""
    successes = 0
    tens = 0
    ones = 0
    miss = 0

    for _ in range(trials):
        roll = np.random.randint(1, 11, num_dice)
        # Count successes: 6-10 = 1 success, 10 = 2 successes
        success_count = np.sum((roll >= 6) & (roll <= 9)) + 2 * np
            .sum(roll == 10)
        ten_count = np.sum(roll == 10)
        one_count = np.sum(roll == 1)

        successes += success_count
        tens += ten_count
        ones += one_count
        if success_count == 0:
            miss += 1

    return {
        'avg_successes': successes / trials,
        'p_at_least_one_success': 1 - (miss / trials),
        'p_at_least_one_ten': tens / (trials * num_dice),  # Per
            die chance
        'p_at_least_one_one': ones / (trials * num_dice),  # Per
            die chance
        'miss_rate': miss / trials
    }
```

```
# Test key dice pools
for dice in [2, 3, 4, 5, 6]:
    stats = simulate_dice_pool(dice)
    print(f"{dice} dice:")
    print(f"  Avg successes: {stats['avg_successes']:.2f}")
    print(f"  P(at least one success): {stats['
       p_at_least_one_success']:.1%}")
    print(f"  P(at least one 10): {stats['p_at_least_one_ten
       ']:.1%}")
    print(f"  P(at least one 1): {stats['p_at_least_one_one
       ']:.1%}")
    print(f"  Miss rate: {stats['miss_rate']:.1%}")
    print()

# Show GM SB generation per player roll
print("GM SB generation per player roll (expected SB = P(at least
   one 1)):")
for dice in [2, 4, 6, 8]:
    stats = simulate_dice_pool(dice)
    print(f"  {dice} dice: {stats['p_at_least_one_one']:.1%} → {
       stats['p_at_least_one_one']:.2f} SB/roll")
print("\nIn a 4-player round:")
for dice in [4, 6]:
    stats = simulate_dice_pool(dice)
    sb_per_round = 4 * stats['p_at_least_one_one']
    print(f"  {dice} dice/player: {sb_per_round:.2f} SB generated
       ")
```